

PyTV Module Design Document

Module:	SUB
Author:	Bolin Li
Date:	2024.11.25
Version:	0.1.0

Revision History:

Version	Author	Date	Notes
0.1.0	Bolin Li	2024.11.25	Delete all redundant contents and descriptions in Sub Module

Release Notes:

There are no release notes for any previous versions.

Contents

1	Preliminaries	1
1.1	Purpose and scope	1
1.2	Notations	1
1.3	Typographic conventions	1
1.4	Dependencies	2
1.4.1	Software dependencies	2
1.4.2	Submodule dependencies	2
2	Functionality	3
2.1	Overview	3
2.2	Configurable parameters	3
2.2.1	Quantization sets	3
2.2.2	Quantization modes	3
2.2.3	Overflow modes	4
2.3	External interface	4
2.3.1	Clock and reset	4
2.3.2	Main Module Interface	4
2.4	Functional description	4
3	Implementation	6
3.1	Overall architecture	6
3.2	Data and control flow	6
3.3	External sequential behavior	6
3.3.1	Latency analysis	6
3.3.2	Throughput analysis	6

1 Preliminaries

1.1 Purpose and scope

This module will realize a general purpose **fixed-point subtraction** operation. It will take two input specified fixed-point value and calculate their subtraction in specified fixed-point format.

1.2 Notations

- LSB: Least significant bit.
- MSB: Most significant bit.
- TCPL: Two's complement positive-only format.
- SMGN: Sign-Magnitude format.
- Functional description: A description of the module's combinational behavior.
- Functional model: A (usually QuBLAS) model of the module's behavior.

1.3 Typographic conventions

Visual Cue	Meaning
(Bullet) •	List of important items.
monospace	Codes or commands typed on the keyboard.
MONOSPACE	Constants, parameters.
<i>italic</i>	Variable values, file and directory names.
<italic>	Placeholder text for users to insert values with.
[item]	Optional items.
bold red text	Important notes, warnings, or errors.
bold green text	Indicating success.
green text	Comments or remarks.
blue text	Revised text or keywords.
monospace/blue	URLs, links.
SMALL CAPS	Hardware submodule names.

1.4 Dependencies

1.4.1 Software dependencies

Software	Version
PyTV (verithon)	0.3
PyTU	0.1.0

1.4.2 Submodule dependencies

This part lists all the PyTV submodule dependencies of the module, including the submodule name and version.

Submodule	Version
DELAY	0.1.0
FxMATCH	0.1.0

2 Functionality

2.1 Overview

The SUB Module receives two input values with configurable precisions, then output their subtraction in a configurable precision fixed-point format. Now the module supports quantization modes and overflow modes listed in section 2.2.2 and 2.2.3, respectively. The module also supports optional **negedge reset** and **enable signal**, which can be configured by IF_RST_N and IF_ENABLE parameters, respectively.

2.2 Configurable parameters

Parameter	Range	Default	Description
QU_IN_1	$[\mathbb{N}^+, \mathbb{Z}, \mathbb{B}]$	[9, 3, True]	Summand data quantization set
QU_IN_2	$[\mathbb{N}^+, \mathbb{Z}, \mathbb{B}]$	[9, 3, True]	Addend data quantization set
QU_OUT	$[\mathbb{N}^+, \mathbb{Z}, \mathbb{B}]$	[9, 3, 1]	Output quantization set
QU_MODE	All available modes in 2.2.2	QuMode.TRN.TCPL	Output FxMatch Quantize Mode
OF_MODE	All available modes in 2.2.3	OfMode.WRP.TCPL	Output FxMatch Overflow Mode
N_PIPELINES	$\mathbb{N}$	0	Number of pipeline stages
IF_RST_N	$\mathbb{B}$	False	Sign for including negedge reset
	$[\mathbb{B}, \dots]$	[False, ...]	Requires N_PIPELINES > 0
IF_ENABLE	$\mathbb{B}$	False	Sign for including enable signal

2.2.1 Quantization sets

Quantization set describes the quantization format of value, which is originated from a PyTU class QuType. Here is a list of QuType's members:

Member	Range	Description
DWT	$\mathbb{N}^+$	Data Width
FRAC	$\mathbb{Z}$	Decimal Point Offset, in bits
IF_SIGNED	$\mathbb{B}$	Signed or unsigned input format

2.2.2 Quantization modes

Quantization mode refers to the method applied when dealing with precision loss (at LSB). This module supported the following modes:

Mode	Description
QuMode.TRN.TCPL	Truncate all extra LSB in TCPL format
QuMode.TRN.SMGN	Truncate all extra LSB in Sign-Magnitude (SMGN) format, then convert back to TCPL format
QuMode.RND.POS_INF	Round to nearest quantization level, when the value is equidistant between two levels, round to the larger value
QuMode.RND.NEG_INF	Round to nearest quantization level, when the value is equidistant between two levels, round to the smaller value
QuMode.RND.ZERO	Round to nearest quantization level, when the value is equidistant between two levels, round to the value closer to zero

<code>QuMode.RND.INF</code>	Round to nearest quantization level, when the value is equidistant between two levels, round to the value further from zero
<code>QuMode.RND.CONV</code>	Round to nearest quantization level, when the value is equidistant between two levels, round to the even value (LSB = 0)

2.2.3 Overflow modes

Overflow mode refers to the method applied when dealing with overflow (at MSB). This module supported the following modes:

Mode	Description
<code>OfMode.WRP.TCPL</code>	Wrap around in TCPL format
<code>OfMode.SAT.TCPL</code>	Saturate to the maximum value in TCPL format
<code>OfMode.SAT.SMGN</code>	Saturate to the maximum value in SMGN format, then convert back to TCPL
<code>OfMode.SAT.ZERO</code>	Saturate to zero

2.3 External interface

2.3.1 Clock and reset

Synchronous reset and asynchronous release is the default reset behavior. If the module has a different reset behavior, it should be described here.

Signal	Width	Description
<code>i_clk</code>	1	Clock signal (Requires <code>N_PIPELINES > 0</code>)
<code>i_rst_n</code>	1	Reset signal (Requires <code>N_PIPELINES > 0</code>)

2.3.2 Main Module Interface

Signal	Width	Description
<code>i_data_1</code>	<code>QU_IN_1.DWT</code>	Input signal for summand data
<code>i_data_2</code>	<code>QU_IN_2.DWT</code>	Input signal for addend data
<code>o_result</code>	<code>QU_OUT.DWT</code>	Output signal for result
<code>i_ctrl_en</code>	1	Enable signal for the Sub module

2.4 Functional description

Here depicts a functional description of the module.

Algorithm 1: Module function

```
Input: i_data_1
Input: i_data_2
Input: i_ctrl_en
Output: o_result

1 if IF_ENABLE == True then
2   data_1 = i_data_1 && i_ctrl_en;
3   data_2 = i_data_2 && i_ctrl_en;
4 else
5   data_1 = i_data_1;
6   data_2 = i_data_2;

  // Minus data_2 for signed/unsigned situations
7 if QU_IN_2.IF_SIGNED == True then
8   // Use Two's Complement
9   data_2_minus = {data_2[QU_IN_2.DWT-1], data_2} + 1'b1;
10 else
11   data_2_minus = {1'b0, data_2} + 1'b1;

  // Calculate Quantization Set for adding from input data
12 DWT_FIX = max(IntBits(QU_IN_1), IntBits(QU_IN_2)+1) + max(FracBits(QU_IN_1),
    FracBits(QU_IN_2)) + 1; // Minus operation add a bit to data_2
13 FRAC_FIX = max(QU_IN_1.FRAC, QU_IN_2.FRAC);
14 SIGN_FIX = QU_IN_1.IF_SIGNED || QU_IN_2.IF_SIGNED;
15 QU_IN_FIX = QuType(DWT_FIX, FRAC_FIX, SIGN_FIX);
16 data_1_fixed = data_1.FxMatch(QU_IN = QU_IN_1, QU_OUT = QU_IN_FIX);
17 data_2_fixed_minus = data_2_minus.FxMatch(QU_IN=QU_IN_2, QU_OUT=QU_IN_FIX);

  // Adder
18 result_unfixed = data_1_fixed + data_2_fixed_minus;

  // Quantize and handle overflow to get the specified-format final result
19 o_result = result_unfixed.FxMatch(QU_IN = QU_IN_FIX, QU_OUT = QU_OUT, QU_MODE = QU_MODE,
    OF_MODE = OF_MODE);
```

3 Implementation

3.1 Overall architecture

The SUB Module relies on the FxMATCH module to perform the fixed-point subtraction operation. The FxMATCH module takes two input values with configurable precisions, then output their sum in a configurable precision fixed-point format. The SUB Module then applies the specified quantization and overflow modes to the output value.

When N_PIPELINES is 0, the module is a combinational module and the output value is directly generated in the combinational logic. When N_PIPELINES is greater than 0, the module is a pipelined module with N_PIPELINES pipeline stages, and the output value is generated in a pipeline stage.

3.2 Data and control flow

The module is either a combinational module (when N_PIPELINES is 0) or a fully pipelined module (when N_PIPELINES > 0).

3.3 External sequential behavior

3.3.1 Latency analysis

Latency are N_PIPELINES clock cycles for output the result.

3.3.2 Throughput analysis

This module can handle one frame per clock cycle.